INFORME DE FIN DE UNIDAD

Datos Generales

Asignatura	Base de datos
Ciclo	3 A
Unidad	1
Resultado de aprendizaje de la unidad	Modela bases de datos de problemas reales, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Nombre del Docente	Edwin René Guamán Quinche
Fecha	09-11-2026
Enlace de Git	
Nombres y apellidos del estudiante	Galo Milton Benítez Cabrera

1. TITULO

Implementación de un sistema web para la Optimización y la Gestión de un sistema de gestión de donaciones y banco de alimentos.

2. CONTEXTO DEL PROBLEMA

Describir de forma detallada el problema o necesidad que motivó el diseño de la base de datos. Esta sección debe responder a las siguientes preguntas:

- **¿Cuál es la situación o entorno donde se presenta el problema?**

  El entorno donde se presenta el problema, es en la gestión de donaciones de alimentos donde se requiere registrar y organizar adecuadamente quién realiza las donaciones, qué productos se reciben, en qué cantidades se encuentran disponibles y a qué personas u organizaciones beneficiarias se entregan. Esta actividad se desarrolla principalmente en bancos de alimentos, fundaciones y organizaciones sociales que reciben aportes de diferentes donantes para posteriormente distribuirlos entre quienes necesitan esta ayuda, de la misma manera también con el manejo de la información relacionada con la recepción, almacenamiento y distribución de productos alimenticios. Sin embargo, cuando estos procesos se realizan mediante registros manuales, hojas de cálculo independientes o documentos físicos, puede resultar complicado mantener la información centralizada, actualizada y relacionada correctamente. Por esta razón, surge la necesidad de diseñar una base de datos que permita administrar los datos de manera organizada y facilite el seguimiento de las operaciones.

- **¿Qué dificultades o necesidades existen actualmente sin una base de datos?**

Una de las principales dificultades que pueden presentarse sin una base de datos centralizada es el control de las donaciones y de los productos disponibles. Puede resultar difícil identificar quién realizó cada donación, qué productos fueron recibidos, en qué cantidades ingresaron y cuántas unidades permanecen disponibles en el inventario. Así mismo, pueden producirse inconsistencias, duplicación de registros o dificultades para consultar la información cuando se necesita.

- **¿A quiénes afecta el problema? (usuarios, institución, organización)**

Esta problemática afecta principalmente a los responsables de administrar las donaciones, controlar el inventario y coordinar las entregas, así también con quienes necesitan disponer de información confiable para desarrollar sus actividades. También involucra a los donantes, cuyos aportes deben registrarse correctamente, y a los beneficiarios, que dependen de una distribución organizada de los alimentos. De igual manera, afecta a la institución u organización, debido a que necesita controlar sus recursos y mantener información clara sobre las operaciones realizadas.

- **¿Qué datos se necesitan gestionar y por qué es importante hacerlo correctamente?**

Para poder manejar los datos de una mejor manera, se debe gestionar los siguientes tipos de información:

- **Donantes y donaciones:** permite registrar los datos de las personas o entidades que realizan aportes, las fechas en que se reciben las donaciones y su estado.
- **Productos, categorías e inventario**: permite identificar los alimentos disponibles, sus nombres, unidades de medida, cantidades, categorías y fechas de actualización del inventario.
- **Beneficiarios y solicitudes:** permite registrar a las personas u organizaciones beneficiarias, sus datos, las fechas de solicitud, los estados y las observaciones correspondientes.
- **Detalles de donaciones y solicitudes:** permite identificar los productos incluidos en cada donación o solicitud, junto con las cantidades correspondientes.
- **Entregas y sus detalles:** permite registrar las fechas de distribución, el estado de las entregas y los productos y cantidades entregados a los beneficiarios.

Y es importante la correcta gestión de estos datos, porque permite mantener un control de los alimentos recibidos y disponibles, realizar un seguimiento de las solicitudes y verificar las entregas efectuadas. Además, facilita las consultas, reduce la repetición innecesaria de información y contribuye a evitar inconsistencias en los registros.

3. OBJETIVOS

3.1. Objetivo General

Diseñar, elaborar e implementar un sistema de gestión de donaciones y banco de alimentos que permita organizar la información de los donantes, productos, inventario, beneficiarios, solicitudes y entregas.

3.2. Objetivos Específicos

- Diseñar el modelo relacional del Sistema de Gestión de Donaciones y Banco de Alimentos, identificando y definiendo las 11 entidades, sus atributos, claves primarias, claves foráneas, relaciones y cardinalidades, mediante la elaboración del diagrama entidad-relación y el esquema relacional.
- Implementar la estructura de la base de datos mediante SQL, creando las 11 tablas con sus respectivas claves primarias y foráneas, incorporando registros de prueba y desarrollando consultas que permitan verificar el almacenamiento, organización y recuperación de la información de donaciones, productos, inventario, beneficiarios, solicitudes y entregas.

4. MODELOS CONCEPTUALES DE BASES DE DATOS

4.1. Modelo Entidad-Relación (E-R)

Insertar en este espacio el diagrama Entidad-Relación (E-R) elaborado para el sistema. El diagrama debe incluir todos los elementos del modelo:

- Entidades (representadas como rectángulos): listar todas las entidades identificadas.
- Atributos (representados como elipses): incluir atributos simples, compuestos, derivados y multivaluados si los hay.
- Relaciones (representadas como rombos): indicar el nombre de la relación entre entidades.
- Cardinalidades: especificar las cardinalidades de cada relación (1:1, 1: N, M: N).
- Llaves primarias: subrayar los atributos que son identificadores de cada entidad.

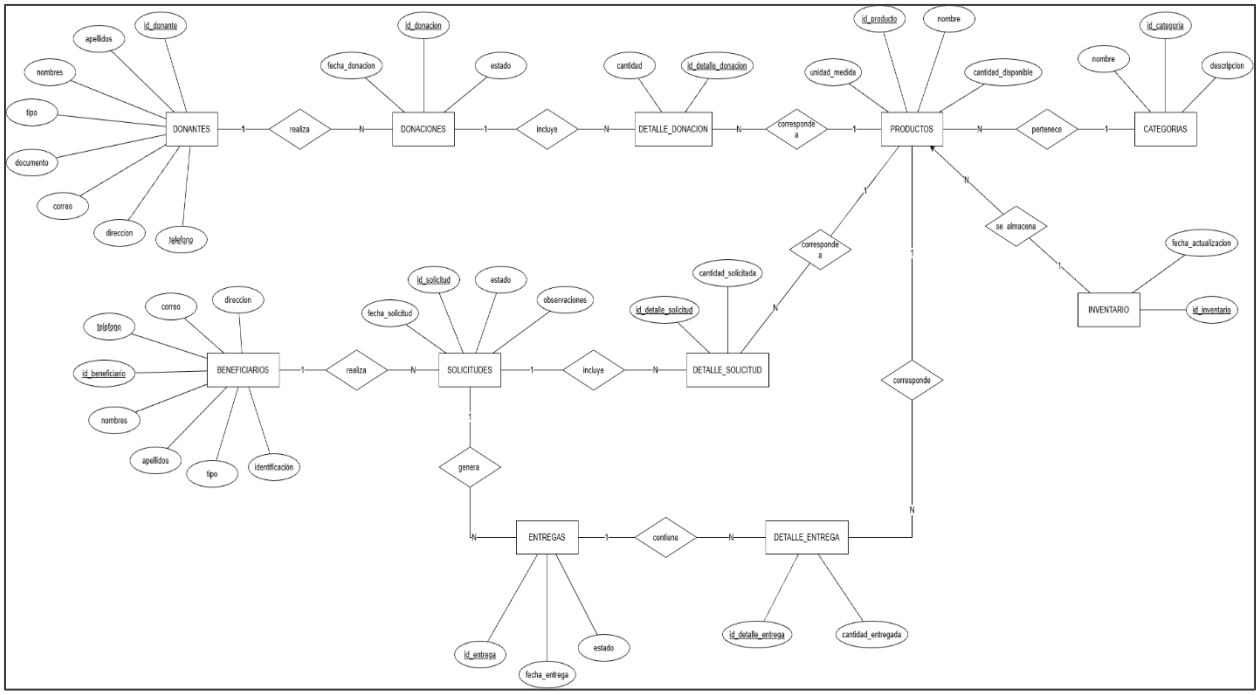


Figura 1. Diagrama Entidad-Relación del sistema {Sistema de Donaciones}.

4.2. Modelo Relacional

Presentar el modelo relacional derivado del diagrama E-R. Este modelo muestra cómo se transforman las entidades y relaciones en tablas con sus respectivas claves. Debe incluir:

- El nombre de cada tabla (relacion).
- Los atributos de cada tabla con su tipo de dato sugerido.
- Indicacion clara de la Clave Primaria (PK) en cada tabla.
- Indicacion de las Claves Foraneas (FK) y a que tabla hacen referencia.

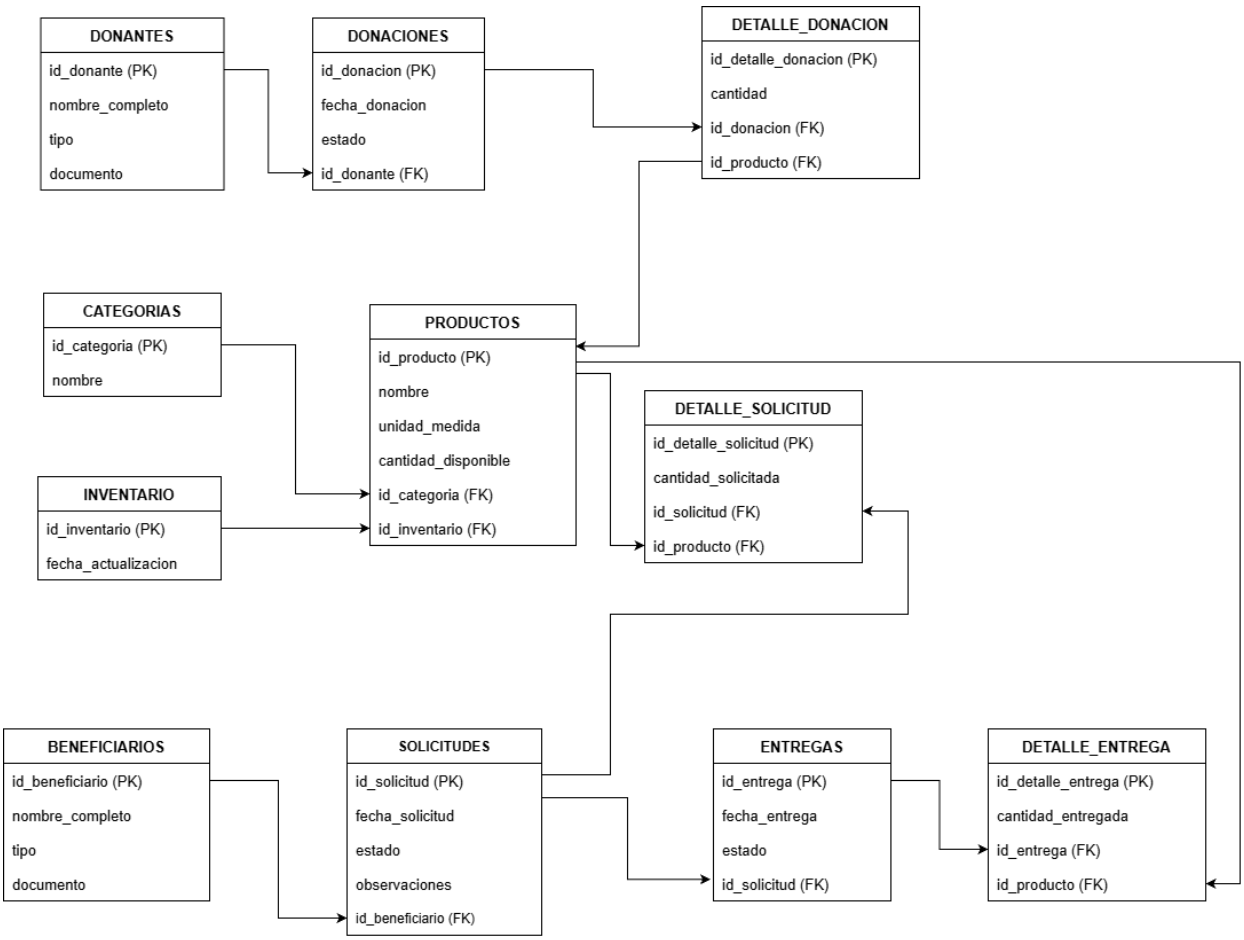


Figura 2. Modelo Relacional del sistema {Sistema de Donaciones}.

CÓDIGO SQL

5.1. Creación de Tablas (DDL)

Presentar el código SQL correspondiente a la creación de al menos 3 tablas representativas del sistema. El código debe incluir:

- Sentencia CREATE TABLE con nombre de tabla en mayúsculas o siguiendo la convención del equipo.
- Definición de columnas con tipos de datos apropiados (INT, VARCHAR, DATE, DECIMAL, etc.).
- Restricciones: PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT, CHECK.
- Claves foráneas con FOREIGN KEY ... REFERENCES para mantener la integridad referencial.

```
-- 1. DONANTES
CREATE TABLE DONANTES (
    id_donante VARCHAR (15) PRIMARY KEY,
    nombre_completo VARCHAR (25) NOT NULL,
    tipo VARCHAR (12) NOT NULL,
    documento VARCHAR (20) UNIQUE
);
-- 2. CATEGORIAS
CREATE TABLE CATEGORIAS (
    id_categoria VARCHAR (15) PRIMARY KEY,
    nombre VARCHAR (30) NOT NULL UNIQUE
);
-- 3. INVENTARIO
CREATE TABLE INVENTARIO (
    id_inventario VARCHAR (15) PRIMARY KEY,
    fecha_actualizacion DATE NOT NULL
);
-- 4. PRODUCTOS
CREATE TABLE PRODUCTOS (
    id_producto VARCHAR (15) PRIMARY KEY,
    id_categoria VARCHAR (15) NOT NULL,
    id_inventario VARCHAR (15) NOT NULL,
    nombre VARCHAR (50) NOT NULL,
```

```
    unidad_medida VARCHAR (20) NOT NULL,
    cantidad_disponible DECIMAL (10,2) NOT NULL DEFAULT 0.00,
    FOREIGN KEY (id_categoria) REFERENCES CATEGORIAS (id_categoria),
    FOREIGN KEY (id_inventario) REFERENCES INVENTARIO (id_inventario)
);
-- 5. DONACIONES
CREATE TABLE DONACIONES (
    id_donacion VARCHAR (15) PRIMARY KEY,
    id_donante VARCHAR (15) NOT NULL,
    fecha_donacion DATE NOT NULL,
    estado VARCHAR (15) NOT NULL,
    FOREIGN KEY (id_donante) REFERENCES DONANTES (id_donante)
);
-- 6. DETALLE_DONACION
CREATE TABLE DETALLE_DONACION (
    id_detalle_donacion VARCHAR (15) PRIMARY KEY,
    id_donacion VARCHAR (15) NOT NULL,
    id_producto VARCHAR (15) NOT NULL,
    cantidad DECIMAL (10,2) NOT NULL,
    FOREIGN KEY (id_donacion) REFERENCES DONACIONES (id_donacion),
    FOREIGN KEY (id_producto) REFERENCES PRODUCTOS (id_producto)
);
-- 7. BENEFICIARIOS
CREATE TABLE BENEFICIARIOS (
    id_beneficiario VARCHAR (15) PRIMARY KEY,
    nombre_completo VARCHAR (25) NOT NULL,
    tipo VARCHAR (12) NOT NULL,
```

```
    documento VARCHAR (20) UNIQUE
);
-- 8. SOLICITUDES
CREATE TABLE SOLICITUDES (
    id_solicitud VARCHAR (15) PRIMARY KEY,
    id_beneficiario VARCHAR (15) NOT NULL,
    fecha_solicitud DATE NOT NULL,
    estado VARCHAR (15) NOT NULL,
    observaciones VARCHAR (100),
    FOREIGN KEY (id_beneficiario) REFERENCES BENEFICIARIOS (id_beneficiario)
);
-- 9. DETALLE_SOLICITUD
CREATE TABLE DETALLE_SOLICITUD (
    id_detalle_solicitud VARCHAR (15) PRIMARY KEY,
    id_solicitud VARCHAR (15) NOT NULL,
    id_producto VARCHAR (15) NOT NULL,
    cantidad_solicitada DECIMAL (10,2) NOT NULL,
    FOREIGN KEY (id_solicitud) REFERENCES SOLICITUDES (id_solicitud),
    FOREIGN KEY (id_producto) REFERENCES PRODUCTOS (id_producto)
);
-- 10. ENTREGAS
CREATE TABLE ENTREGAS (
    id_entrega VARCHAR (15) PRIMARY KEY,
    id_solicitud VARCHAR (15) NOT NULL,
    fecha_entrega DATE NOT NULL,
    estado VARCHAR (15) NOT NULL,
    FOREIGN KEY (id_solicitud) REFERENCES SOLICITUDES (id_solicitud)
```

```
);

-- 11. DETALLE_ENTREGA

CREATE TABLE DETALLE_ENTREGA (

    id_detalle_entrega VARCHAR (15) PRIMARY KEY,

    id_entrega VARCHAR (15) NOT NULL,

    id_producto VARCHAR (15) NOT NULL,

    cantidad_entregada DECIMAL (10,2) NOT NULL,

    FOREIGN KEY (id_entrega) REFERENCES ENTREGAS (id_entrega),

    FOREIGN KEY (id_producto) REFERENCES PRODUCTOS (id_producto)

);
```

5.2. Inserción de Datos (DML)

Incluir sentencias INSERT INTO con datos de ejemplo representativos para al menos 3 tablas. Los datos deben ser coherentes entre sí (las claves foráneas deben corresponder a registros existentes) y deben cubrir casos variados para que las consultas posteriores sean significativas.

```
-- 1. DONANTES
INSERT INTO DONANTES (id_donante, nombre_completo, tipo, documento) VALUES
('D001', 'Ana Torres', 'Persona', '1101000001'),
('D002', 'Luis Castillo', 'Persona', '1101000002'),
('D003', 'Mercado Central', 'Empresa', '1191000001'),
('D004', 'Rosa Jimenez', 'Persona', '1101000004'),
('D005', 'Comercial Loja', 'Empresa', '1191000005');

-- 2. CATEGORIAS
INSERT INTO CATEGORIAS (id_categoria, nombre) VALUES
('C001', 'Granos'),
('C002', 'Enlatados'),
('C003', 'Lacteos'),
('C004', 'Cereales'),
('C005', 'Aceites');

-- 3. INVENTARIO
INSERT INTO INVENTARIO (id_inventario, fecha_actualizacion) VALUES
('I001', '2026-10-01'),
('I002', '2026-10-05'),
('I003', '2026-10-10'),
('I004', '2026-10-15'),
```

```
('I005', '2026-10-20');

-- 4. PRODUCTOS
INSERT INTO PRODUCTOS
(id_producto, id_categoria, id_inventario, nombre, unidad_medida, cantidad_disponible) VALUES
('P001', 'C001', 'I001', 'Arroz', 'Kilogramo', 120.00),
('P002', 'C002', 'I002', 'Atun enlatado', 'Unidad', 80.00),
('P003', 'C003', 'I003', 'Leche', 'Litro', 45.00),
('P004', 'C004', 'I004', 'Avena', 'Kilogramo', 0.00),
('P005', 'C005', 'I005', 'Aceite vegetal', 'Litro', 60.00);

-- 5. DONACIONES
INSERT INTO DONACIONES
(id_donacion, id_donante, fecha_donacion, estado) VALUES
('DN001', 'D001', '2026-10-02', 'Recibida'),
('DN002', 'D002', '2026-10-06', 'Recibida'),
('DN003', 'D003', '2026-10-11', 'Verificada'),
('DN004', 'D004', '2026-10-16', 'Recibida'),
('DN005', 'D005', '2026-10-21', 'Verificada');
-- 6. DETALLE_DONACION
INSERT INTO DETALLE_DONACION
(id_detalle_donacion, id_donacion, id_producto, cantidad) VALUES
('DD001', 'DN001', 'P001', 50.00),
('DD002', 'DN002', 'P002', 30.00),
('DD003', 'DN003', 'P003', 25.00),
('DD004', 'DN004', 'P005', 20.00),
('DD005', 'DN005', 'P001', 40.00);

-- 7. BENEFICIARIOS
INSERT INTO BENEFICIARIOS (id_beneficiario, nombre_completo, tipo, documento) VALUES
('B001', 'Fundacion Esperanza', 'Fundacion', '1192000001'),
('B002', 'Comedor San Jose', 'Comedor', '1192000002'),
('B003', 'Casa Hogar Luz', 'Fundacion', '1192000003'),
('B004', 'Centro Solidario', 'Institucion', '1192000004'),
('B005', 'Comedor La Paz', 'Comedor', '1192000005');

-- 8. SOLICITUDES
INSERT INTO SOLICITUDES
(id_solicitud, id_beneficiario, fecha_solicitud, estado, observaciones) VALUES
('S001', 'B001', '2026-10-03', 'Aprobada', 'Solicitud mensual'),
('S002', 'B002', '2026-10-07', 'Aprobada', 'Atencion a familias'),
('S003', 'B003', '2026-10-12', 'Pendiente', 'Apoyo alimentario'),
('S004', 'B004', '2026-10-17', 'Aprobada', NULL),
('S005', 'B005', '2026-10-22', 'Aprobada', 'Entrega parcial');

-- 9. DETALLE_SOLICITUD
```

```
-- Todos los beneficiarios solicitan arroz (P001); se agregan otros productos
-- para que las consultas de interseccion, diferencia y division sean demostrables.
INSERT INTO DETALLE_SOLICITUD
(id_detalle_solicitud, id_solicitud, id_producto, cantidad_solicitada) VALUES
('DS001', 'S001', 'P001', 10.00),
('DS002', 'S001', 'P002', 5.00),
('DS003', 'S002', 'P001', 15.00),
('DS004', 'S002', 'P003', 8.00),
('DS005', 'S003', 'P001', 12.00),
('DS006', 'S003', 'P004', 4.00),
('DS007', 'S004', 'P001', 20.00),
('DS008', 'S004', 'P005', 6.00),
('DS009', 'S005', 'P001', 10.00),
('DS010', 'S005', 'P002', 3.00);

-- 10. ENTREGAS
INSERT INTO ENTREGAS
(id_entrega, id_solicitud, fecha_entrega, estado) VALUES
('E001', 'S001', '2026-10-04', 'Completada'),
('E002', 'S002', '2026-10-08', 'Completada'),
('E003', 'S003', '2026-10-13', 'Pendiente'),
('E004', 'S004', '2026-10-18', 'Completada'),
('E005', 'S005', '2026-10-23', 'Parcial');

-- 11. DETALLE_ENTREGA
INSERT INTO DETALLE_ENTREGA
(id_detalle_entrega, id_entrega, id_producto, cantidad_entregada) VALUES
('DE001', 'E001', 'P001', 10.00),
('DE002', 'E001', 'P002', 5.00),
('DE003', 'E002', 'P001', 15.00),
('DE004', 'E002', 'P003', 8.00),
('DE005', 'E003', 'P001', 0.00),
('DE006', 'E004', 'P001', 20.00),
('DE007', 'E004', 'P005', 6.00),
('DE008', 'E005', 'P001', 5.00),
('DE009', 'E005', 'P002', 3.00);
```

CONSULTA DE ÁLGEBRA RELACIONAL Y TRADUCIR A SQL

Presentar 5 consultas formuladas primero en Álgebra Relacional y luego traducidas al lenguaje SQL. Las consultas deben tener diferente nivel de complejidad (relación de dos tablas o más) e involucrar distintos operadores del álgebra relacional.

Operadores del algebra relacional a utilizar: *Seleccion (sigma), Proyeccion (pi), Union, Interseccion, Diferencia, Producto cartesiano, Reunion natural (JOIN), Division. Cada consulta debe usar al menos un operador diferente.*

Consulta 1 — Selección y proyección (σ y π)

Descripción del objetivo: Mostrar el nombre, la unidad de medida y la cantidad disponible de los productos que tienen existencias mayores que cero.
Álgebra Relacional:

```
π nombre, unidad_medida, cantidad_disponible (σ cantidad_disponible > 0 (PRODUCTOS))
```

SQL:

```
SELECT nombre, unidad_medida, cantidad_disponible
FROM PRODUCTOS
WHERE cantidad_disponible > 0;
```

Resultado esperado: Se mostrarán los productos que tienen existencias disponibles. En los datos de ejemplo aparecerán arroz, atún enlatado, leche y aceite vegetal; la avena no aparecerá porque su cantidad disponible es cero.

Consulta 2 — Unión (∪)

Descripción del objetivo: Obtener una lista conjunta de nombres y tipos de personas u organizaciones que participan en el sistema como donantes o beneficiarios. Ambas proyecciones tienen los mismos atributos y tipos compatibles.
Álgebra Relacional:

```
π nombre_completo, tipo (DONANTES) ∪ π nombre_completo, tipo (BENEFICIARIOS)
```

SQL:

```
SELECT nombre_completo, tipo
FROM DONANTES
UNION
SELECT nombre_completo, tipo
FROM BENEFICIARIOS;
```

Resultado esperado: Se obtendrá una sola lista con los donantes y beneficiarios. UNION elimina filas duplicadas, si existieran.

Consulta 3 — Intersección (∩) y diferencia (−)

Descripción del objetivo: Identificar los productos que aparecen tanto en los detalles de donaciones como en los detalles de solicitudes, y también los productos solicitados que todavía no aparecen en ningún detalle de donación.
Álgebra Relacional:

```
Productos presentes en ambos conjuntos:\nπ id_producto (DETALLE_DONACION) ∩ π id_producto (DETA-
LLE_SOLICITUD)\n\nProductos solicitados pero no donados:\nπ id_producto (DETALLE_SOLICITUD) − π
id_producto (DETALLE_DONACION)
```

SQL:

```
-- Intersección: productos que fueron donados y solicitados.
SELECT DISTINCT dd.id_producto
FROM DETALLE_DONACION dd
WHERE dd.id_producto IN (
   SELECT ds.id_producto
   FROM DETALLE_SOLICITUD ds
);

-- Diferencia: productos solicitados que no aparecen en donaciones.
SELECT DISTINCT ds.id_producto
FROM DETALLE_SOLICITUD ds
WHERE ds.id_producto NOT IN (
   SELECT dd.id_producto
   FROM DETALLE_DONACION dd
);
```

Resultado esperado: La intersección devolverá los identificadores de productos presentes en ambos conjuntos. La diferencia devolverá los productos solicitados que no constan en los detalles de donación. Con los datos de ejemplo, P004 será un producto solicitado que no fue donado.

Consulta 4 — Producto cartesiano (×) con selección (σ)

Descripción del objetivo: Relacionar cada categoría con los productos que pertenecen a ella. Primero se forma el producto cartesiano entre CATEGORIAS y PRODUCTOS y luego se filtran las parejas cuyos identificadores de categoría coinciden.
Álgebra Relacional:

π CATEGORIAS.nombre, PRODUCTOS.nombre (σ CATEGORIAS.id_categoria = PRODUCTOS.id_categoria (CATEGORIAS × PRODUCTOS))

SQL:

```
SELECT c.nombre AS categoria,
      p.nombre AS producto
FROM CATEGORIAS c
CROSS JOIN PRODUCTOS p
WHERE c.id_categoria = p.id_categoria;
```

Resultado esperado: Se mostrará el nombre de cada categoría junto con el producto asociado. No se mostrarán combinaciones que no correspondan a la categoría real del producto.

Consulta 5 — Reunión (JOIN) y agrupamiento (γ)

Descripción del objetivo: Calcular la cantidad total entregada de cada producto, mostrando el nombre del producto y sumando las cantidades registradas en los detalles de entrega.
Álgebra Relacional:

γ nombre; SUM(cantidad_entregada) (PRODUCTOS ⋈ PRODUCTOS.id_producto = DETALLE_ENTREGA.id_producto DETALLE_ENTREGA)

SQL:

```
SELECT p.id_producto,
```

```
    p.nombre,
    SUM(de.cantidad_entregada) AS total_entregado
FROM PRODUCTOS p
JOIN DETALLE_ENTREGA de
  ON p.id_producto = de.id_producto
GROUP BY p.id_producto, p.nombre
ORDER BY p.nombre;
```

Resultado esperado: Se obtendrá una fila por producto que tenga detalles de entrega, con la suma de las cantidades entregadas. La consulta permite conocer qué productos se han distribuido en mayor o menor cantidad.

CONCLUSIONES

Redactar las conclusiones del proceso de implementación de la base de datos trabajada durante la unidad. Las conclusiones deben ser reflexivas, fundamentadas en la experiencia práctica y relacionadas directamente con los objetivos planteados.

Estructura recomendada para las conclusiones:

- **Conclusión 1. Modelado conceptual y relacional.** La elaboración del diagrama entidad-relación y del modelo relacional permitió identificar los elementos principales del Sistema de Gestión de Donaciones y Banco de Alimentos, definiendo sus 11 entidades, atributos y relaciones. Una de las partes más importantes fue establecer correctamente las claves primarias, claves foráneas y cardinalidades, especialmente en las relaciones entre donaciones, productos, solicitudes y entregas. Este proceso permitió organizar la información de manera lógica, reducir la duplicidad de datos y establecer una estructura adecuada para el funcionamiento de la base de datos.
- **Conclusión 2. Implementación mediante SQL**. La creación de las tablas mediante sentencias DDL y la inserción de registros mediante DML permitió comprender cómo transformar el diseño relacional en una estructura de base de datos. Durante este proceso, el uso de claves primarias y foráneas fue fundamental para mantener la integridad referencial y asegurar que los registros relacionados correspondieran a datos existentes. Asimismo, se comprobó la importancia de crear las tablas e insertar los registros respetando el orden de las dependencias entre ellas.
- **Conclusión 3. Álgebra Relacional y consultas SQL.** El desarrollo de consultas mediante Álgebra Relacional y su posterior traducción a SQL permitió comprender cómo recuperar, combinar y analizar la información almacenada. La aplicación de operadores como selección, proyección, unión, intersección, diferencia, producto cartesiano y división permitió representar diferentes necesidades de consulta. Además, el uso de JOIN y funciones de agrupamiento facilitó la obtención de información relacionada y resultados resumidos sobre las donaciones, los productos, las existencias, las solicitudes y las entregas. Esta actividad permitió reconocer la importancia de las consultas para aprovechar correctamente los datos almacenados en el sistema.

BIBLIOGRAFÍA

Registrar todas las fuentes consultadas para el desarrollo del informe siguiendo el formato IEEE

[1] “Recovery and redistribution of food | Technical Platform on the Measurement and Reduction of Food Loss and Waste | Food and Agriculture Organization of the United Nations,” *FoodLossWaste*, Sep. 05, 2024. https://www.fao.org/platform-food-loss-waste/food-waste/food-waste-reduction/recovery-and-redistribution-of-safe-and-nutritious-food-for-human-consumption/en
[2] Banco de alimento Diakonia, “Inicio | Diakonia,” *Diakonia*, Mar. 27, 2026. https://bancodealimentosdiakonia.org/
[3] “Google Books: Find, preview, and read books online.” https://books.google.com/books/about/Fundamentals_of_Database_Systems.html